Potential of LLMs in Cyber Incident Detection and Response Systems

Ing. Juraj Paška
Faculty of Electrical Engineering and Information Technology (FEI)
Slovak University of Technology in Bratislava (STU)
Bratislava, Slovakia
juraj.paska@stuba.sk

***Abstract*—Modern Security Operations Centers (SOCs) face significant operational hurdles, including overwhelming alert fatigue caused by rule-based systems, manual incident response bottlenecks, and the difficulty of processing vast amounts of unstructured threat intelligence.**

This paper investigates the potential of Large Language Models (LLMs) in cyber incident detection and response systems. This paper also presents an implementation of Multiagent Incident Response Assistant (MIRA), a framework designed to streamline the incident reaction phase through a specialized multi-agent orchestration layer and hybrid Retrieval-Augmented Generation (RAG). MIRA utilizes a supervisor-based architecture to delegate tasks among specialized sub-agents: a Todo Agent for workflow planning, an Action Agent for technical command generation, and a Chat Agent for analyst interaction. We implement a hybrid search mechanism combining BM25 full-text search with semantic cosine similarity to ensure responses are grounded in verified security playbooks.

Our evaluation employs a "LLM-as-Judge" approach with the Mistral-3 (14B) model, benchmarks the performance of Qwen3 (14B), Llama 3.1 (8B), and Phi-4 (3.8B) for three types of tasks. Results show potential of RAG in technical tasks but also limitations in conversational tasks due to verbosity caused by long and unstructured playbooks retrieved from memory.

***Index Terms*—LLM, Cybersecurity, SIEM, SOC, RAG, Incident Detection, SOAR**

I. Introduction

Security Operations Centers (SOCs) serve as the foundation of enterprise cybersecurity defense. The ideal SOC operates through structured tiers of personnel—Tier 1 alert analysts, Tier 2 incident responders, and Tier 3 threat hunters and senior analysts. SOC operators are using security tools including Security Information and Event Management (SIEM), Security Orchestration, Automation, and Response (SOAR), and Endpoint Detection and Response (EDR). However, the real-world execution of these operations is limited by amount of manual work of SOC operators. Classical SIEM systems are rule-based, generating overwhelming volume of false positives that inevitably lead to alert fatigue for Tier 1 analysts. Furthermore, incident response processes remain manual, creating operational bottlenecks during critical mitigation phases. A significant hurdle in modern SOC operations is also unstructured threat intelligence. Critical threat intelligence, such as Common Vulnerabilities and Exposures (CVEs) or MITRE ATT&CK matrices, exists primarily as unstructured text, making it difficult for teams to analyse and extract Indicators of Compromises (IoCs) and SIEM rules.

LLMs present an interesting research area to address these core limitations. In this paper, we investigate the potential of LLMs in cybersecurity operations with focus on improving threat intelligence analysis, incident detection and response. The domains of integration of LLMs into cybersecurity operations we may split into following directions:

1) **Incident Detection:** Addressing false positives and alert fatigue, prioritize critical alerts, and explain complex logs by augmenting them with relevant historical and environmental context.
2) **Incident Response:** Reduce amount of manual work needed to mitigate incidents. We propose Multiagent Incident Response Assistant (MIRA) for this purpose.
3) **Threat Intelligence Analysis:** Analysis of cyber threat intelligence and translate it into actionable SIEM detection rules.
4) **Business Code Analysis:** LLMs show potential in analyzing business source code to identify vulnerabilities. This paper does not cover this direction.

The remainder of this paper is organized as follows. Section II-A describes an empirical study on analyst interactions with LLMs. Sections II-B through II-F examine applications of LLMs in SIEM and detection. Section II-G explores advanced RAG frameworks, followed by an analysis of digital forensics in Section II-H. Sections II-I through II-K discuss autonomous agents for cyber threat intelligence (CTI) analysis. Section III presents the architecture and methodology of our proposed framework, the Multiagent Incident Response Assistant (MIRA). Section IV details our experimental evaluation and results. Section V discusses current limitations and potential for future research. Section VI provides concluding remarks.

II. Related Work and Context

A. Analysis of Analyst Interaction with LLMs

Recent research suggests that SOC analysts uses LLMs primarily as support tool rather than as autonomous experts. According to a study [1] that analyzed thousands of analyst-generated queries in an operational environment, LLMs are most frequently used for command explanation, scriptwriting, documentation improvement, and data analysis support.

The experiment described in [1] was conducted by capturing all queries sent to the GPT-4 model. A total of over 3,000 queries from 45 analysts were recorded over a 10-month period. A key finding was the growing trend in the frequency of LLM usage in the daily workflow of a SOC analyst, with 11 out of 45 analysts integrating GPT-4 into their everyday routine.

The research showed that only 4% of queries involved a request to directly evaluate an alert as malicious or benign, confirming the use of LLMs as assistive rather than decision-making tools. Furthermore, 75% of interactions consisted of only 2 to 3 questions, mostly involving iterative text refinement for incident reporting or explanations of functions and logs. The authors highlight the potential for integrating LLM tools directly into SIEM systems to reduce cognitive load by eliminating context switching.

B. Multi-model Approaches and Classification for Reducing False Positives

In [2], the authors explored a multi-model approach to classify security incidents into two categories: interesting and uninteresting, where interesting alerts are supposed to be evaluated by SOC operator and uniteresting alerts are marked as false positives. The highest success rate was achieved using GPT-4, reaching a precision of 94% and a recall of 92%. According to the authors, implementation of this approach led to a 40% reduction in incident processing time and a 60% decrease in cognitive load due to fewer manually handled alerts.

The experimental framework proposed by the researchers consists of a modular architecture featuring an Alert Generation Module, an LLM Agent, and a Reporting Module. This system was integrated into an environment utilizing the Wazuh SIEM platform, where CALDERA was employed for adversary emulation to generate realistic attack scenarios. This setup allowed for a comparative benchmark between five distinct models: GPT-4, GPT-3.5, LLaMA 3, Mixtral 8x22B, and OpenHermes 2.5 Mistral 7B. The inclusion of open-source models like LLaMA and Mixtral was intended to address critical privacy risks and data sovereignty concerns associated with processing sensitive SOC data through external APIs.

Despite the strong performance metrics of the proprietary models, the study highlights a significant operational hurdle: a hallucination rate of approximately 5% during the analysis of security events. While GPT-4 remained the top performer with an F1-score of 93%, the open-source alternatives demonstrated varying degrees of efficacy, trailing behind in complex classification tasks. This performance gap underscores the challenge of balancing model accessibility and privacy with the high accuracy required for autonomous alert triage.

To address these limitations, the authors advocate for a hybrid "co-pilot" strategy where LLMs act alongside human analysts. They also propose further research into prompt optimization and bias detection to improve the reliability of the system.

Despite these benefits, the study has limitations. The reported accuracy figures show performance of individual models, while the success rate of the multi-model voting itself is not clearly specified. Another open point is redundancy; if GPT-4 and GPT-3.5 are highly correlated, the third model's output may be consistently overruled. Furthermore, the nature of the "tuning" mentioned is unclear—whether it involved full fine-tuning or merely adjusting weights for final scoring—and.

C. Integration into Open-Source SIEM

The authors in [3] present the Security Event Response Copilot (SERC) system, built on the open-source Wazuh SIEM. The system adds two components: a RAG-based module for context-aware incident retrieval and an LLM module for generating recommendations. The RAG system draws information from MITRE ATT&CK, the NIST Cybersecurity Framework (CSF), and the SOCFortress incident response knowledge base.

The efficacy of the framework was evaluated through an end-to-end simulation using the Atomic Red Team tool to replicate six critical threat categories, including ransomware, malware, and phishing. To assess the quality of the generated responses, the researchers employed multi-faceted metrics such as cosine similarity for semantic alignment and BERTScore for contextual accuracy.

The study compares the performance of Pixtral and OpenAI models. Pixtral demonstrated high stability and consistency accross all types of incidents, while OpenAI showed greater adaptability but higher performance variability. Pixtral consistently achieved higher precision and F1-scores (ranging between 85.4% and 87.2%) across diverse attack scenarios, reflecting its reliability in capturing relevant information without significant fluctuations. In contrast, while the OpenAI model peaked at higher semantic alignment (75.81% cosine similarity in certain cases), it exhibited performance dips, suggesting that Pixtral may be a more stable candidate for consistent automated triage in open-source SIEM environments.

D. Predictive Threat Intelligence Models

Reference [4] investigates the integration of LLMs to overcome the limitations of static, signature-based detection systems that frequently struggle with zero-day exploits and polymorphic threats. The authors introduce the Predictive Threat Intelligence Model (PTIM), a framework trained on approximately 100 million security-related data points, including network logs and public threat databases.

Empirical validation shows that PTIM achieves a detection accuracy exceeding 95% with a substantial reduction in false positives compared to rule-based systems. Authors also claim that PTIM performs significantly faster than standard LLM architectures, what could be an advantage during reaction-time critical ransomware attacks.

E. LLM-Based Agents for Tier 1 Triage Automation

Thesis [5] provides a specific evaluation of LLM agents within the critical triage process of Tier 1 SOC analysts. The

study proposes a structured framework where an LLM agent serves as an intermediary between a SIEM platform (Wazuh) and the human analyst. By testing five models including OpenAI's GPT-4o, Meta's Llama 3, and Mixtral 8x22B—the research investigates the agents' ability to classify alerts as either "interesting" or "not-interesting" based on raw log data. This classification logic is intended to filter out the high volume of "noise" generated by normal user operations, allowing analysts to focus on high-fidelity threats. Security incidents were emulated using adversary simulation tool CALDERA.

Experimental findings indicate that LLM agents significantly enhance the speed of initial triage and reduce the cumulative cognitive load on security staff by providing natural language explanations for each alert classification. Despite these gains, the research highlights critical limitations such as the occasional hallucinations and the inherent privacy risks of processing sensitive log data through external models. Consequently, the study concludes that while LLM agents are highly effective for routine automation, they are best deployed in a "co-pilot" capacity. This hybrid approach ensures that human expertise remains in the loop to verify LLM reasoning.

F. Multi-Agent Modular Architectures for Correlated Threat Detection

Isolated detection systems often fail to correlate related events across different layers of an organization's infrastructure. To address this, reference [6] proposes a modular multi-agent architecture that integrates domain-specific cybersecurity tools with the semantic reasoning capabilities of Large Language Models. The framework consists of three specialized autonomous agents—focused on email verification, server log analysis, and IP range scanning—which independently process data using tailored detection pipelines. These agents translate raw technical outputs into structured risk signals, which are then fed into a centralized contextual recommendation system.

By cross-analyzing signals (e.g., correlating a suspicious phishing email with subsequent lateral movement in server logs), the system can detect complex threat patterns that evade standalone sensors. According to the study, experimental results on benchmark datasets, including CIC-IDS 2017 and SpamAssassin, demonstrate a high detection accuracy of 93.6% and a 41.3% reduction in false positives compared to traditional rule-based methods. Furthermore, the use of LLM-based Chain-of-Thought (CoT) reporting provides explicable insights for security analysts, significantly reducing triage time and increasing confidence in the system's automated recommendations.

G. Advanced RAG Frameworks

Reference [7] addresses the amount of unstructured cybersecurity information which are relevant to daily work of SOC analyst. The proposed MoRSE framework is an open-source system integrating dual RAG modules in a cascade: a structured RAG for pre-processed data and an unstructured RAG for raw text. By drawing from CVE repositories, MITRE, and ExploitDB, MoRSE achieved a 15% improvement in general question accuracy and a 50% increase in accuracy for CVE-related incidents compared to GPT-4 without RAG.

The technical distinction of MoRSE (Mixture of RAGs Security Experts) lies in its parallel retriever architecture, which was inspired by the Mixture of Experts (MoE) paradigm to handle multidimensional cybersecurity contexts. The framework operates in two distinct phases: an Information Retrieval phase, where specialized retrievers collect semantically related data in varied formats, followed by an Answer Generation phase driven by an LLM.

H. LLMs in Digital Forensics and Timeline Analysis

Beyond real-time detection and threat intelligence, post-incident investigation—specifically Digital Forensics and Incident Response (DFIR)—presents a significant bottleneck for security teams. Constructing an accurate timeline of a cyber incident is a complex, labor-intensive task that requires parsing thousands of heterogeneous security logs to identify correlations. The authors in [8] address this challenge by introducing GenDFIR, a framework that leverages Retrieval-Augmented Generation (RAG) combined with Large Language Models to automate cyber incident timeline analysis.

By ingesting raw system events (such as Windows Security event logs detailing failed logons, credential validation errors, and audit log clearing) into a dynamic vector knowledge base, GenDFIR enables an LLM to accurately piece together the sequence of adversary actions. This methodology overcomes the traditional limitations of manual log triage, significantly accelerating the reconstruction of attack paths while mitigating human error and analyst fatigue. The RAG architecture is crucial in this context, as it anchors the LLM's outputs strictly to the ingested forensic artifacts, thereby preventing hallucinations and maintaining the evidentiary integrity of the analysis.

Evaluations of the GenDFIR framework demonstrated its capability to successfully reconstruct complex scenarios, such as unauthorized access chains, by maintaining strict chronological coherence and contextual relevance. By automating the extraction and chronological mapping of crucial artifacts, the system produces timeline reports that are directly applicable for formal incident reporting, root-cause analysis, and the formulation of subsequent defensive strategies.

I. Autonomous Agents for Cyber Threat Intelligence

The automation of Cyber Threat Intelligence (CTI) analysis represents a critical frontier in reducing the manual workload of SOC analysts. As explored in [9] and [10], security teams traditionally rely on unstructured reports from sources such as CrowdStrike or MITRE ATT&CK to manually derive correlation rules for SIEM systems. This process is slow and prone to human error. To address this, recent research proposes an autonomous AI agent capable of parsing natural language reports to identify Indicators of Compromise (IOCs) and automatically generating valid regular expressions (RegEx) for direct SIEM integration without human supervision.

The proposed system uses a graph-based approach to visualize relationships between IOCs, such as connecting specific file hashes to the domains and IP addresses involved in a singular attack campaign. However, achieving full autonomy requires overcoming four primary challenges: ensuring factual accuracy to filter hallucinations, translating complex natural language into syntactically correct RegEx, performing automated verification of these patterns, and maintaining the structural integrity of IOC relationship graphs.

Experimental results underscore the potential of this approach. In an analysis of over 50 CTI reports, the model identified approximately 2,900 IOCs. After autonomous filtering, 2,300 valid indicators remained, from which 2,200 RegEx patterns were successfully generated. Remarkably, the system missed only 3% of valid IOCs compared to manual ground truth.

J. Optimization of SIEM Detection Rule Sets

While the automated generation of rules significantly expands detection coverage, it often introduces a secondary challenge: the proliferation of redundant or overlapping rules. Such overlaps increase computational overhead, response latency, and, most critically, alert fatigue for SOC analysts. Reference [11] introduces RuleGenie, a LLM-aided recommender system designed to optimize enterprise-level SIEM rule sets. The authors argue that manual optimization is increasingly unsustainable due to the sheer scale of modern rule bases, which can contain thousands of platform-specific queries (e.g., KQL, Sigma, or SPL) with complex logic.

The RuleGenie framework leverages the multi-head attention capabilities of transformer models to generate high-dimensional embeddings of SIEM rules. By applying a similarity matching algorithm to these embeddings, the system identifies the top-k most similar rules within a repository. Furthermore, RuleGenie utilizes Chain-of-Thought (CoT) reasoning to analyze the underlying logic of flagged rules, allowing it to recommend consolidations or deletions of redundant logic. This optimization process ensures that the SIEM maintains a lean and efficient detection posture, reducing the signal-to-noise ratio and ensuring that the most relevant alerts are prioritized for human investigation.

K. Automated Mapping of SIEM Rules to Adversarial Techniques

While the generation and optimization of SIEM rules are vital for visibility, the effectiveness of a security posture is often measured by its alignment with standardized frameworks like MITRE ATT&CK. As explored in [12], manual mapping of structured detection rules—such as those written in Splunk's Search Processing Language (SPL)—to specific Tactics, Techniques, and Procedures (TTPs) is a complicated process prone to error. To resolve this, the authors propose Rule-ATT&CK Mapper (RAM), an approach using Large Language Models to autonomously categorize structured rules. RAM first translates technical rule syntax into natural language descriptions enriched by external contextual information (e.g., IoC significance) before performing the final technique recommendation.

Experimental evaluations using the Splunk Security Content dataset demonstrate that RAM improves mapping accuracy without the need for model fine-tuning. By testing multiple models, including GPT-4-Turbo and IBM Granite, the study found that incorporating a web-search agent to retrieve supplementary contextual knowledge about Indicators of Compromise (IoCs) is essential for overcoming the limitations of an LLM's implicit domain knowledge. The framework achieved an F1-score of 0.62 and a recall of 0.75, highlighting its capability to assist SOC teams in maintaining an up-to-date and accurate coverage map of the ATT&CK matrix.

III. MIRA System Architecture

To address the limitations of traditional Security Operations Centers (SOC), we propose prototype of the Multiagent Incident Response Assistant (MIRA). MIRA is designed as framework to assist analysts during the incident reaction phase by leveraging specialized LLM agents and RAG with hybrid search. This framework is built as part of Team Project of students at STU FEI.

A. System Architecture and Tech Stack

The MIRA framework is built upon a modular infrastructure deployed in "on-premises" environment. The core orchestration is handled via *LangGraph*, which enables a multiagent architecture, while *Chainlit* provides the user interface for human-operator interaction. The backend is powered by *Django* and *PostgreSQL*, the latter specifically utilized for storing embeddings and performing vector operations such as semantic search using cosine similarity and fulltext search using algorithm BM25.

The system integrates a playbook database consisting of approximately 90 playbooks across various security categories, managed via *MediaWiki* and *Cacao Roaster*. For local model execution and management, *Ollama* is employed, ensuring that sensitive security data can be processed on-premises or within restricted hardware environments.

B. Hybrid Search and RAG Methodology

The core of the MIRA framework is a Retrieval-Augmented Generation (RAG) pipeline that leverages a hybrid search approach to maximize the relevance of retrieved security playbooks. This system distinguishes between two primary search approaches:

- **Full-text Search (FT):** This branch focuses on explicit technical identifiers, such as specific CVE IDs, attack categories, or unique security keywords. We implement the **BM25 (Best Matching 25)** algorithm, which ranks documents based on the appearance frequency of search terms relative to the entire playbook repository.
- **Semantic Search (SEM):** This branch enables retrieval based on the underlying intent and context of the query. Security playbooks are vectorized in their entirety with the `all-MiniLM-L6-v2` embedding model and stored

in an vector database. Queries are also vectorized using the same model to ensure consistent similarity scoring. To calculate the relationship between the query vector and playbook embeddings, **cosine similarity** is used.

C. Result Fusion and Configuration

The retrieval system is configurable, with the primary parameter K defining the total number of returned records. To integrate the results from the FT and SEM branches, we consider three distinct fusion methodologies:

1) **Naive Approach:** A simple division of the parameter K in a fixed ratio (e.g., $K_{FT} = 2, K_{SEM} = 3$). This serves as a baseline to test the impact of each method without complex mathematical merging.
2) **Weighted Average:** Results from both branches are first normalized into the interval $[0, 1]$. The final relevance score is calculated as:

$$Score = (W_{SEM} \cdot Norm_{SEM}) + ((1 - W_{SEM}) \cdot Norm_{FT}) \quad (1)$$

where W_{SEM} is a configurable weight (defaulting to 0.5) that balances the influence of semantic context against keyword precision.
3) **Reciprocal Rank Fusion (RRF):** As illustrated in our framework's optimization phase, we utilize RRF to combine the rankings from multiple search algorithms without requiring score normalization. The fused score for a document d within the set of all documents D is calculated as:

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)} \quad (2)$$

where R is the set of rankings (full-text and semantic), $r(d)$ is the rank of document d in ranking r, and k is a constant (typically set to 60) used to mitigate the impact of low-ranked outliers.

In MIRA framework, we implemented approach of Weighted Average, where we retrieve TOP K results via both semantic and fulltext search. Results are then after removal of duplicates merged and TOP K results are returned back from search engine.

D. Testing of search algorithms and Evaluation Framework

To validate the effectiveness of our retrieval, the fusion logic and search parameters K, W_{SEM}, we considered a set of validation questions with expected answer as ground truth. Due to simplicity and size of our dataset we simply considered setting K to value 5 and W_{SEM} to value 0.5. In case of increasing amount of dataset, or with need of splitting dataset into smaller chunks, we propose these approaches for finetuning search engine:

- **Order-based Score:** A scoring system where a retrieved target playbook is assigned points based on its position ($5 - IDX$ for the TOP 5 results).
- **Hit Rate:** A binary metric indicating whether the desired playbook is present within the top K results.
- **Precision@K:** The proportion of relevant playbooks found within the top K retrieved items.
- **Mean Reciprocal Rank (MRR):** The average of the reciprocal ranks of the first relevant playbook, prioritizing configurations that place the correct solution at the top of the list.

E. Testing Models – LLM as Judge

To identify the most effective models for our multi-agent architecture, we developed a testing framework designed to quantitatively evaluate LLM performance within the cybersecurity domain. The framework is built with a focus on modularity, allowing usage of various models via *Ollama*, the adjustment of system prompts, and configuration of RAG parameters.

1) Testbed Architecture and Configuration: The evaluation system utilizes *Dependency Injection* to integrate the hybrid search module defined in previous sections. This allows the testing script to call a standardized interface and an option to connect different search engine if necessary.

By decoupling the search logic from the model evaluation, we can measure what is impact of RAG on perfomance of system. For this study, we evaluated three different models in two primary states: (1) **Plain LLM (Zero-shot)**, where the model relies solely on its internal training data, and (2) **LLM + RAG**, where the model's prompt is extended with relevant playbook segments retrieved from our database.

2) Evaluation Metrics and Agent Roles: We define specific success criteria based on the three functional sub-agents within the MIRA architecture. To determine the most effective configuration, each underlying model was tested in two states: as a standalone LLM and as an LLM augmented with RAG. The evaluation focuses on the following roles:

- **Supervisor Evaluation:** We measure the model's ability to categorize an incident correctly and map the analyst's query to the appropriate sub-agent (Todo, Action, or Chat). The primary metric is classification accuracy.
- **Sub-agent Evaluation:**
  - **Todo Agent:** Evaluated on its ability to break down complex security incidents into logical, sequential steps based on playbook requirements.
  - **Action Agent:** Assessed on its ability to generate concrete technical commands and generalize entities—for example, correctly identifying and replacing a placeholder IP address from a template with the specific IP found in the incident log.
  - **Chat Agent:** Evaluated on its ability to provide concise, context-aware explanations and maintain a coherent dialogue with the SOC analyst.

F. Multiagent Orchestration

MIRA uses a "Supervisor" architecture to manage task delegation. In this paradigm, a central supervisor receives the analyst's query, identifies the intent, and manages the handoff to the specialized worker agents. We define our primary sub-agents as follows:

- **Todo Agent:** Responsible for workflow management and the generation of structured task lists (To-Dos) for incident resolution.
- **Action Agent:** Acts as the technical generator, drafting specific response instructions and commands based on identified playbooks.
- **Chat Agent:** Serves as the conversational interface, allowing analysts to query the system for context, summaries, or clarification on specific alerts.

Architecture (Figure 1) maintains logical separation between agents, allowing for specialized context management. The supervisor agent closes the loop and provides answer back to the operator. We aim to configurable implementation of agents to enable option of configuring specific LLM for specific agent.

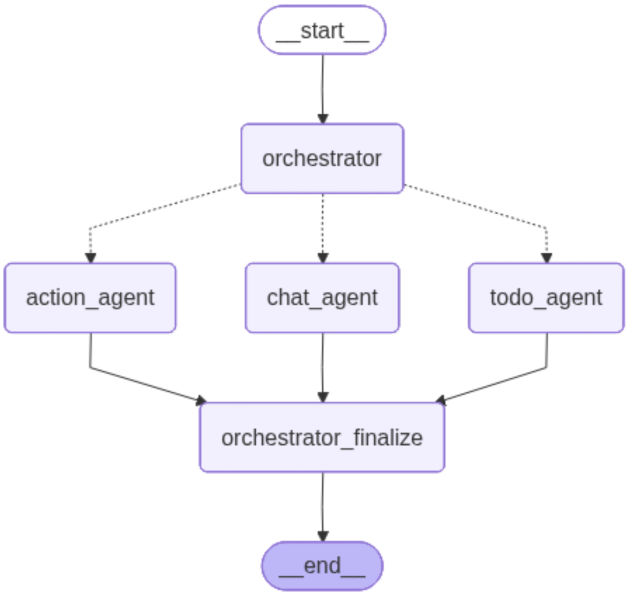

Fig. 1. Multiagent orchestration in MIRA

G. Infrastructure and CI/CD

The development lifecycle is supported by a CI/CD pipeline integrated with GitLab. Every merged Pull Request triggers an automated build of a Docker image for the Python application, which is then deployed to the "on-premises" server via SSH. This ensures seamless further development and testing, since framework is still in its development phase.

IV. Experimental Results and Analysis

To determine the optimal configuration for the MIRA framework, we conducted a series of benchmarks comparing three Large Language Models—**Qwen3 (14B)**, **Llama 3.1 (8B)**, and **Phi-4 (3.8B)**—across our three specialized agent roles. The evaluation was split into baseline performance (without RAG) and augmented performance (with RAG). The results, summarized in Table I, highlight the varying impact of retrieval-augmented context on different task types.

For each scenario, there were 10 prompts for each agent. After generation of responses, we used another LLM (*Mistral-3*) as a judge to evaluate the responses with given validation criteria. Judge LLM provided validation comment and score between 0 and 1 for each response, indicating, how well the response met the criteria.

TABLE I
Average performance scores by Agent and Model

Agent	Model	Without RAG	With RAG
Chat Agent	Qwen3 (14B)	**0.86**	0.56
	Llama 3.1 (8B)	0.61	0.45
	Phi-4 (3.8B)	0.58	0.63
Action Agent	Llama 3.1 (8B)	0.00	**0.31**
	Phi-4 (3.8B)	0.02	0.25
	Qwen3 (14B)	0.00	0.17
Todo Agent	Qwen3 (14B)	0.25	**0.32**
	Phi-4 (3.8B)	0.24	0.26
	Llama 3.1 (8B)	0.23	0.17

The experimental data reveals several critical insights regarding the behavior of LLMs in a SOC environment:

A. The Necessity of RAG for Technical Tasks

For the **Action Agent**, performance without RAG was negligible, with scores ranging from 0.00 to 0.02. Without access to the specific syntax and parameters defined in the security playbooks, the models consistently failed to produce valid technical commands. By enabling RAG, the **Llama 3.1 (8B)** model showed the most significant improvement, reaching a score of 0.31.

B. The Verbosity Penalty in Conversational Tasks

A surprising trend was observed in the **Chat Agent** role, particularly with the **Qwen3** and **Llama 3.1** models. Both models saw a decrease in performance when RAG was enabled. The reasoning provided by the *Mistral-3* judge indicated a "verbosity penalty"; the inclusion of retrieved playbook context often led the models to generate responses that were overly detailed or included unnecessary procedural information, drifting away from the directness required in a conversational setting. Only the **Phi-4 (3.8B)** model managed to improve its chat score with RAG (0.58 to 0.63).

Analysis: As seen in Table II, the score dropped because the model attempted to be overly helpful with the retrieved context. The judge (Mistral-3) penalized the response for failing the "conciseness" and "relevance" criteria, highlighting that for simple definitions, RAG can introduce noise.

V. Limitations and Future Work

While the MIRA framework demonstrates the potential for multi-agent automation in the SOC, several limitations remain that provide opportunities for future research. Also it is important to note the fact, that MIRA framework is a proof of concept and we will continue to develop it in the future. We state limitations of framework below.

TABLE II
CHAT AGENT COMPARISON (QUERY: "WHAT IS A RANSOMWARE NOTE?")

Feature	Baseline (No RAG)	RAG-Augmented
Validation Criteria	Directness, accuracy, and avoidance of unnecessary remediation advice.	
Expected Answer	A message left by attackers stating files are encrypted and providing payment instructions.	
Actual Answer	Provides a concise and accurate definition of a ransomware note and typical delivery methods.	Provides the definition but follows with the entire SOC ransomware workflow and recovery steps.
Validation Comment	"Direct, accurate, and context-aware answer with no unnecessary guidance."	"Response is more detailed than necessary... drifts into unrelated remediation advice."
Score	**1.0**	**0.3**

A. Evaluation Methodology

- **Evaluation Framework:** Our current assessment relies exclusively on the *LLM-as-Judge* paradigm. While efficient, future work should incorporate hybrid evaluation metrics, including **BERTscore** and **Cosine Similarity**, alongside qualitative feedback from expert human analysts to provide a more holistic validation of agent performance. Research on correlation between LLM-as-Judge evaluation and human evaluation would also help to evaluate how well LLM-as-Judge evaluates agent performance. In addition to that, including automated validation of syntax of instructions and commands generated by agent would improve the evaluation process.
- **Dataset Alignment:** The evaluation dataset included several queries that fell outside the scope of the available playbook repository. While this served to highlight the necessity of RAG, it likely depressed the performance scores of baseline models. Future benchmarks should utilize a more balanced dataset to measure the limits of both zero-shot and retrieval-augmented reasoning.
- **Hyperparameter Tuning of LLM calls:** This study did not explore the impact of **temperature settings** on agent performance. Given that temperature significantly influences the balance between technical precision and conversational creativity, future iterations will include a grid search to identify optimal hyperparameter configurations for each specialized agent.
- **Hyperparameter Tuning of hybrid search:** This study did not explore the impact of setting of parameters **K - TOP K playbooks retrieved** and **weight of semantic search** on RAG performance. Future iterations will include a grid search to identify optimal hyperparameters configurations for different types of search query.

B. Architecture and Data Management

- **Granularity of Vectorization:** Currently, security playbooks are vectorized in their entirety. This coarse-grained approach can lead to the inclusion of irrelevant information in the context window, potentially inducing noise or hallucinations. Moving to a more granular, step-level chunking strategy could improve retrieval precision and agent focus.
- **Context Management:** We use approach of sliding window with appended RAG context to manage dialogue context. While functional, this may be suboptimal for complex, multi-stage incidents. Future research will explore more sophisticated memory architectures, such as **Vector-based Long-term Memory** or **Hierarchical Summarization**, to maintain coherence over extended investigations.
- **Supervisor Logic:** The current supervisor functions primarily as an intent classifier for task delegation. Enhancing this agent with **Reasoning Loops (ReAct)** and autonomous validation steps would allow the system to handle more ambiguous security scenarios that require multi-step deductive reasoning.
- **Multiagent vs Singleagent:** This paper would also benefit with comparison of performance between single agent and multiagent approach. Using single agent would simplify an architecture what would result in reduced implementations costs.
- **Using existing agentic solutions:** With increasing popularity of solution like Hermes or OpenClaw, it would be worth to compare performance of similar system designed in these tools.

C. Optimization and Learning

- **Prompt Engineering:** Performance remains highly sensitive to the structure and phrasing of system prompts. Further optimization using techniques such as **Chain-of-Thought (CoT)** or **Few-shot prompting** specifically tailored to cybersecurity ontologies could yield significant gains in accuracy.
- **Feedback Loops and Model Training:** The current framework does not incorporate a mechanism for learning from past interactions. We plan to implement a **continuous feedback loop** where human-analyst corrections are used to refine model weights over time, potentially through **Reinforcement Learning from AI/Human Feedback (RLAIF/RLHF)** to adapt the system to specific organizational environments.

VI. CONCLUSION

In this work, we presented the prototype of the Multiagent Incident Response Assistant (MIRA), framework designed to

accelerate incident response times and improve quality of responses.

Our investigation into the MIRA framework yielded several insights into the integration of AI within cybersecurity:

- **RAG application for technical tasks:** Our experimental results showed that Retrieval-Augmented Generation improves accuracy for technical security tasks. Without RAG, all evaluated LLMs failed to generate valid command-line instructions or JSON-formatted actions. The implementation of a hybrid search approach (combining BM25 and semantic cosine similarity) may provide technical details to LLM to allign with specific tools used in SOC. For further improvements of RAG approach, splitting playbooks into smaller chunks may be proposing approach for potentially better results.
- **RAG application for conversational tasks:** On the other hand, results showed that RAG can degrade performance in conversational tasks. The models tend to generate overly detailed or unnecessary procedural information when provided with retrieved context, which may reduce the quality of human-analyst interaction.
- **Threshold for RAG:** Results showed especially bad performance for questions, which were not grounded in our dataset. In these scenarios, system prompt was augmented with context outside of scope of question and degraded quality of final answer. This fact needs to be considered in future work.

Future work will focus on limitations described in Section V.

Acknowledgment

This framework was built as part of a Team Project by students at the Faculty of Electrical Engineering and Information Technology, Slovak University of Technology in Bratislava (STU FEI).

Appendix A
Supplementary Material: LLM Validation Results

Detailed validation criteria and raw evaluation results for the tested agents are provided in the supplementary material file (`llm_validation_results.xlsx`). This supplementary dataset contains six sheets detailing the exact evaluation criteria, agent interactions, and performance metrics used during the benchmarking phase, split by agent and RAG configuration:

- **Action Agent Validation**: Criteria and outcomes for baseline performance (without RAG) and augmented performance (with RAG).
- **Todo Agent Validation**: Criteria and outcomes for baseline performance (without RAG) and augmented performance (with RAG).
- **Chat Agent Validation**: Criteria and outcomes for baseline performance (without RAG) and augmented performance (with RAG).

References

[1] X. Zhang, Y. Li, and H. Chen, "An empirical study on soc analyst interaction with large language models," arXiv preprint arXiv:2508.18947, 2025, online. Available: https://arxiv.org/abs/2508.18947.

[2] R. Singh, A. Patel, and S. Kumar, "A multimodel approach for enhancing siem performance using llms," Research Square, 2025, online. Available: https://www.researchsquare.com/article/rs-5615639/v1.

[3] L. García and M. Torres, "Integrating ai agents into open-source siem platforms: A wazuh-based framework," *Sensors*, vol. 25, no. 3, p. 870, 2025, online. Available: https://www.mdpi.com/1424-8220/25/3/870.

[4] B. Bokkena, "Enhancing it security with llm-powered predictive threat intelligence," in *Proc. 2024 5th International Conference on Smart Electronics and Communication (ICOSEC)*, 2024, pp. 751–756.

[5] O. Oniagbi, "Evaluation of llm agents for the soc tier 1 analyst triage process," Master's thesis, Dept. of Computing, Univ. of Turku, Turku, Finland, 2024, available: https://www.utupub.fi/handle/10024/177651.

[6] Y. Hmimou, M. Tabaa, A. Khiat, and Z. Hidila, "A multi-agent system for cybersecurity threat detection and correlation using large language models," *IEEE Access*, vol. 13, pp. 31 751–31 766, 2025.

[7] D. Kim, L. Zhao, and J. Park, "Morse: Multi-dimensional cybersecurity reasoning via dual rag frameworks," arXiv preprint arXiv:2407.15748, 2024, online. Available: https://arxiv.org/pdf/2407.15748v1.

[8] F. Y. Loumachi, M. C. Ghanem, and M. A. Ferrag, "Advancing cyber incident timeline analysis through retrieval-augmented generation and large language models," *Computers*, vol. 14, no. 2, p. 67, 2025, online. Available: https://www.mdpi.com/2073-431X/14/2/67.

[9] P. Tseng *et al.*, "Autonomous cyber threat intelligence reporting and ioc extraction via large language models," arXiv preprint arXiv:2407.13093, 2024.

[10] ——, "Poster: Towards autonomous ai agents for soc analyst tasks," Proc. IEEE Symposium on Security and Privacy (S&P), 2025, online. Available: https://sp2025.ieee-security.org/downloads/posters/sp25posters-final24.pdf.

[11] A. Shukla, P. A. Gandhi, Y. Elovici, and A. Shabtai, "Rulegenie: Siem detection rule set optimization," arXiv preprint arXiv:2505.06701, 2025.

[12] S. Wudali, A. Shukla, P. A. Gandhi, Y. Elovici, and A. Shabtai, "Rule-att&ck mapper (ram): Mapping siem rules to ttps using llms," arXiv preprint arXiv:2502.02337, 2025.